



**SIMATS**  
ENGINEERING



**SIMATS**  
Saveetha Institute of Medical And Technical Sciences  
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

## Lab Experiment – 13

### Minimax Algorithm for Gaming

#### Aim

To implement the **Minimax algorithm** for a two-player game using Python.

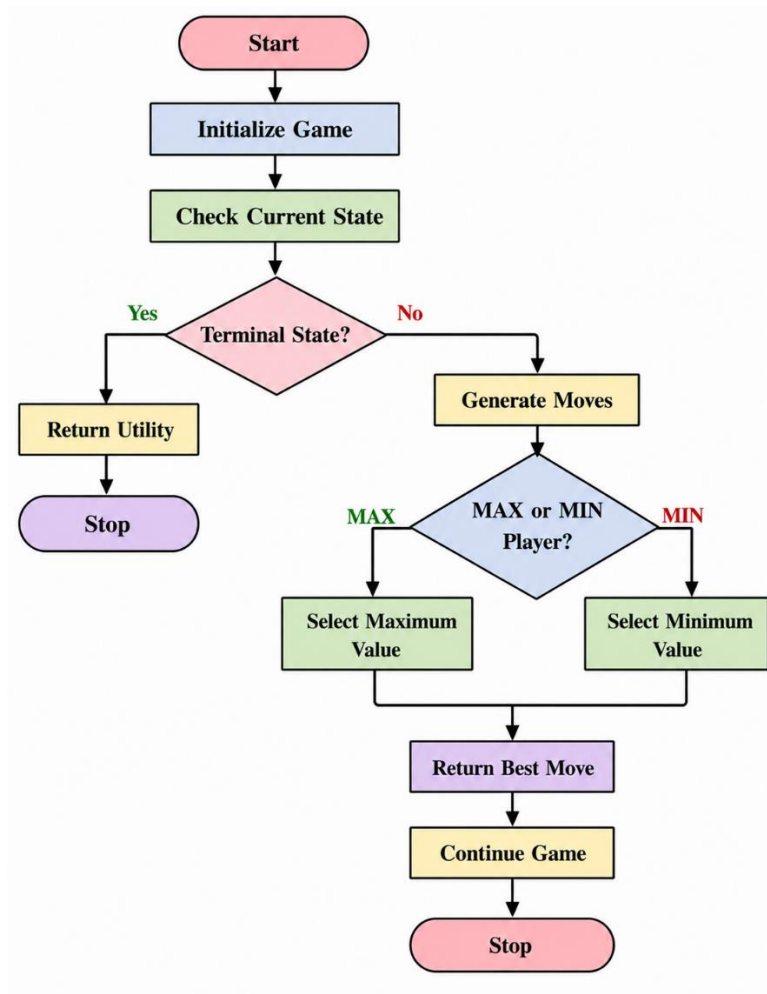
#### Objective

To determine the best possible move for a player by considering all possible game states and assuming that the opponent also plays optimally.

#### Algorithm

1. Start the game.
2. Check whether the current game state is a terminal state.
3. If the game is over, return its utility value.
4. Generate all possible moves.
5. For the **MAX player**, select the move with the highest value.
6. For the **MIN player**, select the move with the lowest value.
7. Recursively evaluate all possible moves.
8. Select the best move for the current player.
9. Display the optimal move.
10. Stop.

## FlowChart



## Code

```
def minimax(depth, node_index, maximizing_player, values):  
    # Terminal node  
    if depth == 3:  
        return values[node_index]  
  
    if maximizing_player:  
        return max(  
            minimax(depth + 1, node_index * 2, False, values),  
            minimax(depth + 1, node_index * 2 + 1, False, values)  
        )
```

else:

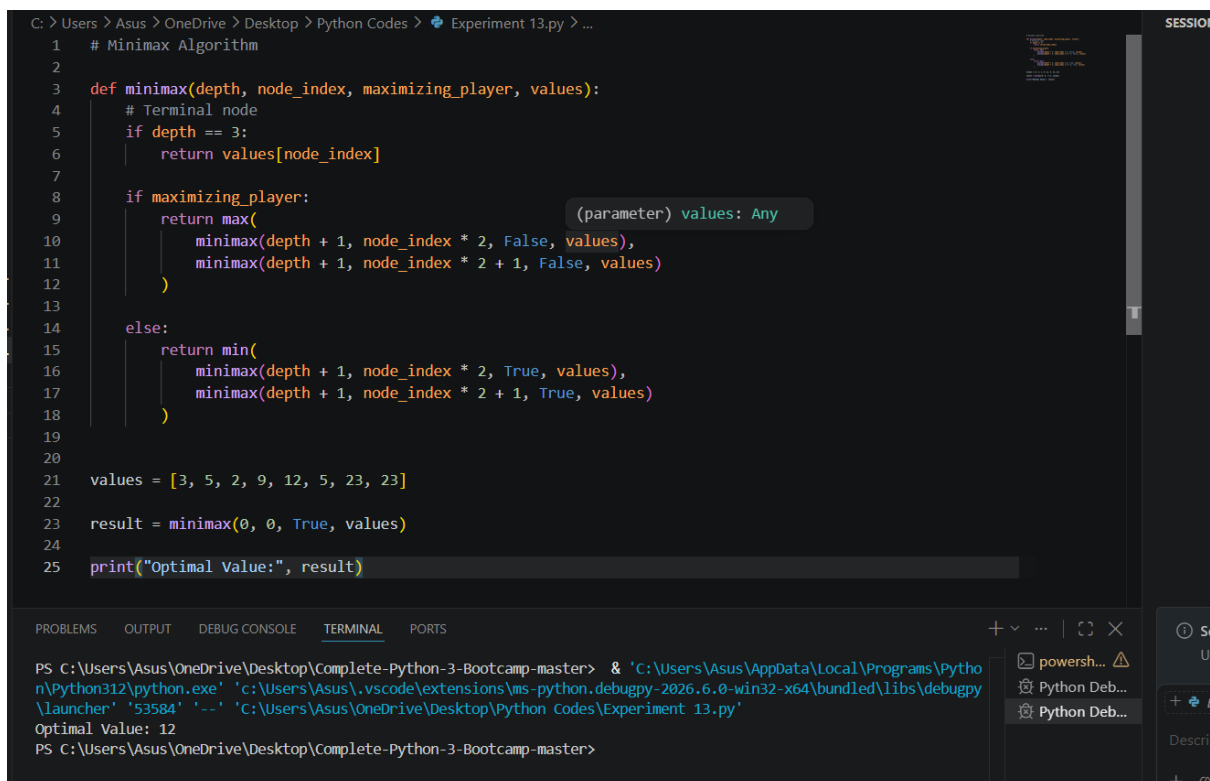
```
    return min(
        minimax(depth + 1, node_index * 2, True, values),
        minimax(depth + 1, node_index * 2 + 1, True, values)
    )
```

```
values = [3, 5, 2, 9, 12, 5, 23, 23]
```

```
result = minimax(0, 0, True, values)
```

```
print("Optimal Value:", result)
```

## Output



The screenshot shows a VS Code editor with a Python file named 'Experiment 13.py'. The code implements a Minimax algorithm for a game tree. The terminal output shows the execution of the script, which prints 'Optimal Value: 12'.

```
C: > Users > Asus > OneDrive > Desktop > Python Codes > Experiment 13.py > ...
1  # Minimax Algorithm
2
3  def minimax(depth, node_index, maximizing_player, values):
4      # Terminal node
5      if depth == 3:
6          return values[node_index]
7
8      if maximizing_player:
9          return max(
10             minimax(depth + 1, node_index * 2, False, values),
11             minimax(depth + 1, node_index * 2 + 1, False, values)
12         )
13
14     else:
15         return min(
16             minimax(depth + 1, node_index * 2, True, values),
17             minimax(depth + 1, node_index * 2 + 1, True, values)
18         )
19
20
21 values = [3, 5, 2, 9, 12, 5, 23, 23]
22
23 result = minimax(0, 0, True, values)
24
25 print("Optimal Value:", result)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\Asus\OneDrive\Desktop\Complete-Python-3-Bootcamp-master> & 'C:\Users\Asus\AppData\Local\Programs\Python\Python312\python.exe' 'c:\Users\Asus\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundled\libs\debugpy\launcher' '53584' '--' 'C:\Users\Asus\OneDrive\Desktop\Python Codes\Experiment 13.py'
Optimal Value: 12
PS C:\Users\Asus\OneDrive\Desktop\Complete-Python-3-Bootcamp-master>
```

## Result

The Minimax algorithm was successfully implemented to determine the optimal value for the given game tree.

## Conclusion

The Minimax algorithm evaluates possible future game states and chooses the best move for the current player, assuming that the opponent will also make the best possible move. It is commonly used in two-player games such as Tic-Tac-Toe and Chess.